



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра алгоритмических языков

Борисов Тимофей Вячеславович

Реализация головоломки-игры sudoku на языке Haskell

Отчёт по практическому заданию

Преподаватели:

Елисеева М.А.

Уразов С.О.

Шамаева Е.Д.

Москва, 2026

Оглавление

<i>Введение.....</i>	<i>3</i>
<i>Постановка задачи</i>	<i>4</i>
<i>Анализ предметной области</i>	<i>5</i>
<i>Описание средств разработки</i>	<i>6</i>
<i>Проектирование программы и архитектура</i>	<i>7</i>
Общее описание модулей.....	7
Структура world.....	7
<i>Разделение логики интерфейса.....</i>	<i>9</i>
<i>Реализация программы</i>	<i>10</i>
Основные структуры данных (Types.hs).....	10
Реализация загрузки уровней (Levels.hs)	10
Реализация представления доски (Board.hs)	11
Реализация генерации новых головоломок (Generator.hs).....	11
Как выполняется рандомизация.....	12
Реализация отрисовки интерфейса (UI.hs)	12
Реализация обработки событий в приложении (Events.hs).....	13
Реализация решателя головоломок (Solver.hs)	14
Реализация ассетов и тем оформления (Assets.hs)	14
<i>Тестирование программы</i>	<i>16</i>
Проверка сборки	16
Проверка интерфейса	16
Проверка игровой логики	16
Проверка подсказки и решателя	16
Проверка генерации	17
Проверка некорректных ситуаций	17
<i>Визуализация игрового процесса</i>	<i>18</i>
<i>Использование технологий искусственного интеллекта</i>	<i>28</i>
Обсуждение структуры программы	28
Обсуждение основных стратегий решения.....	28
Обсуждение работы интерфейса.....	28
Рефакторинг кода	29
Помощь в улучшении генерации уровней.....	29
<i>Заключение</i>	<i>30</i>
<i>Литература.....</i>	<i>31</i>

Введение

Судoku – одна из самых известных логических головоломок, которая обладает простыми и понятными правилами. Процесс игры состоит в последовательном заполнении пустых полей поля размером 9x9. Существуют уровни разной сложности, с разным изначальным заполнением клеток цифрами от 1 до 9. Игроку требуется заполнить все пустые поля таким образом, чтобы в каждый ход новая введённая игроком цифра от 1 до 9 не конфликтовала с остальными существующими на поле. Не должно быть конфликтов (повторений этой цифры) ни в соответствующей вертикали, ни в горизонтали, ни в квадратном блоке 3x3 для данной клетки, которая содержится во всех перечисленных «блоках». Если игрок сможет заполнить поле до конца, головоломка считается решённой.

С помощью языка Haskell можно реализовать все необходимые проверки для игры, определить типы данных для хранения поля и заполненных значений, а также можно осуществить реализацию алгоритмов решения соответствующих головоломок и графического интерфейса для наглядного взаимодействия с данными. Для графической реализации было удобно использовать дополнительную библиотеку Gloss.

Постановка задачи

Целью практического задания была реализация одной из головоломок на выбор (судоку) с использованием графического интерфейса на языке Haskell. Было необходимо реализовать выбор головоломок из файла (заранее предустановленных), возможность взаимодействия с полем, графическое отображение ходов и в целом всего визуального интерфейса, чтобы для пользователя была возможность проходить головоломку. Для каждого хода требовалась проверка его корректности. Дополнительно должна была быть возможность давать подсказку при нажатии на кнопку, а также запуск автоматического решателя головоломки.

Дополнительно были поставлены следующие задачи: улучшения вида интерфейса (добавление иконок для кнопок, плавных анимаций, возможности менять тему приложения, а также достижение аккуратного расположения кнопок в каждом состоянии интерфейса); расширение набора предустановленных головоломок; осуществление псевдо-генерации новых случайных уровней; последовательный запуск различных стратегий решения для каждой головоломки.

Таким образом, в проекте требовалось решить одновременно задачи, связанные с логикой игры, алгоритмами поиска решения, работой с графикой и проектированием архитектуры программы.

Анализ предметной области

Как было сказано выше, для проверки корректности хода необходимо определить, повторяется ли цифра для заданной клетки в соответствующих ей трёх «блоках». С точки зрения программной реализации это означает, что для каждой пустой клетки можно вычислить множество допустимых цифр-кандидатов. На основе этих кандидатов можно как проверять корректность ходов пользователя, так и строить алгоритмы решения.

Помимо реализации самой логики проверки, есть и другие задачи, которые необходимо реализовать. Во-первых, помимо самих проверок важно реализовать всю внутреннюю логику программы: удобные структуры данных для хранения наборов чисел, функции для перевода хода игрока в конкретное изменение состояние доски. Также нужна реализация функций, которые будут загружать заготовленные уровни, и функции, которые взаимодействуют с «миром» (о концепции мира – далее), задавая определенные состояния.

Во-вторых, помимо проработки логики программы, важна полная реализация графического интерфейса, которая сделает возможным:

- непосредственное взаимодействие пользователя с данными;
- ввод ходов;
- графический вывод состояний для пользователя (о конфликте, о том, что головоломка решена и т.д.);
- запуск решателя;
- выбор конкретных уровней;
- запуск псевдо-генерации уровня;
- визуально подсвечивать ошибки пользователя;

И другие задачи.

Таким образом, требуется грамотно определить модули программы, для упрощения и наглядности разработки. Эти модули должны реализовывать две ключевые составляющие программы: работу с графикой и работу со всей логикой игры. При этом интерфейс должен быть понятным и читаемым.

Описание средств разработки

Для реализации использовался язык Haskell в рамках соответствующего практического задания. При этом стоял выбор библиотеки для работы с графикой. Несмотря на несколько вариантов для разработки, была выбрана библиотека Gloss за счет её простоты, отсутствия перегруженности и наличия всех основных методов для отрисовки интерфейса. Хотя с помощью данной библиотеки было бы проблематично работать с различными шрифтами и более тонко настраивать интерфейс, для разработки такой не сложной с точки зрения визуальной составляющей игры как sudoku её оказалось вполне достаточно.

Углубляясь в возможности библиотеки Gloss, можно выделить основные её аспекты, которые были полезны при разработке:

- Открытие графического окна приложения;
- Возможность рисовать примитивы (прямоугольники для кнопок) и текст;
- Загрузка ассетов для кнопок приложения;
- Возможность обрабатывать ввод с клавиатуры и мыши;
- Способность обновлять интерфейс мира.

При создании структур данных, можно выделить две библиотеки, которые немного упростили их представление:

- ``Vector`` для компактного хранения доски и удобного обращения к ней;
- ``Set`` для работы с множествами кандидатов и конфликтов;
- генераторы случайных чисел из стандартной библиотеки для выбора и псевдогенерации уровней.

Проектирование программы и архитектура

Общее описание модулей

Для удобства разработки и наглядности было решено создать несколько модулей программы. Каждый модуль отвечает за свою часть общей системы. Были использованы следующие модули:

- ``Main.hs`` - точка входа в программу;
- ``Types.hs`` - описание основных типов данных;
- ``Config.hs`` - константы и параметры интерфейса;
- ``Board.hs`` - функции для работы с доской и проверки правил;
- ``Levels.hs`` - загрузка встроенных уровней, работа с ними;
- ``Generator.hs`` - генерация новых головоломок (псевдо-генерация);
- ``Solver.hs`` - алгоритмы решения;
- ``Assets.hs`` - загрузка иконок интерфейса;
- ``UI.hs`` - отрисовка экранов;
- ``Events.hs`` - обработка пользовательских событий и изменение состояния.

Структура world

Данная структура отвечает за связку всех модулей программы воедино. В ней хранятся все текущие состояния приложения. В том числе, в них включаются: состояние игры; состояние интерфейса; набор доступных головоломок; текущий seed для генерации; загруженные графические ассеты.

Состояние игры включает в себя:

- `gsInitial` - исходная доска уровня, после загрузки уровня;
- `gsCurrent` - текущее состояние доски;
- `gsGivens` - маска исходных, неизменяемых клеток.

В состояние интерфейса входят следующие состояния:

- `uiScreen` - текущий экран (какой экран сейчас видит пользователь);
- `uiSelected` - выделенная клетка;
- `uiHoverCell` - клетка под курсором;
- `uiHoverButton` - кнопка под курсором;
- `uiTheme` - выбранная тема (есть светлая и тёмная);
- `uiStatus` - текст состояния;
- `uiSolved` - признак завершения sudoku;
- `uiErrorCell` и `uiErrorAlpha` - подсветка ошибочной клетки;
- `uiSolvedAlpha` - анимация успешного решения;
- `uiSolveQueue` - очередь шагов решателя;
- `uiSolveTimer` - таймер между автоматическими ходами.

Примечание: параметры `alpha` использовались для плавного затухания выделения у клеток доски.

При этом, состояние самого world суммарно включает в себя:

- worldGame - игровая часть программы;
- worldUI - графическая часть программы;
- worldLevels - встроенные уровни;
- worldRandomSeed - текущий начальный сид генерации;
- worldAssets - иконки и графические ресурсы.

Таким образом, работа программы заключается в том, чтобы циклически преобразовывать все вышеуказанные состояния.

Разделение логики интерфейса

Как было упомянуто выше, модульность позволила разделить программу на различные части, каждая из которых отвечает за конкретные аспекты общей программы. Формализуя вышесказанное, на этапе проектирования было важно разделить:

- игровую логику и взаимодействие с доской;
- логику решателей головоломок;
- отрисовку интерфейса;
- обработку событий;
- конфигурационные значения в отдельный файл.

Таким образом, удалось добиться того, что правила sudoku не зависят от интерфейса, сам интерфейс не содержит сложной логики решателей, а отдельные конфигурационные параметры (например, размеры, отступы и цвета) были вынесены в модуль `Config.hs`. Это позволило упростить дальнейшее взаимодействие с кодом.

Реализация программы

После разбиения программы на модули и после описания ключевых моментов для работы, ниже будут приведены описания основных элементов для каждого модуля. Из-за большого объема программы, далее будут указаны лишь основные составляющие.

Основные структуры данных (Types.hs)

Для описания программы были введены отдельные типы:

- ``Digit`` - цифра sudoku (ограничение: целое число от 1 до 9);
- ``Cell`` - клетка поля (ограничение: от 1 до 81);
- ``Board`` - доска (вектор длины 81);
- ``Difficulty`` - сложность (Easy, Medium, Hard);
- ``Screen`` - текущий экран интерфейса;
- ``ButtonAction`` - действие кнопки;
- ``SolverStrategy`` - стратегия решения;
- ``GameState`` - игровое состояние;
- ``UIState`` - состояние пользовательского интерфейса;
- ``World`` - полное состояние приложения.

Примечание 1: для отсутствия конфликтов, уменьшения ошибок и общей структурности использовалось задание новых типов с помощью ``newtype``, вместо использования ``type`` и ``data``.

Примечание 2: структура `Vector` в дальнейшем позволила более удобным образом обращаться к элементам доски, добавляя собой компактность и возможность удобно преобразовывать индексы.

Реализация загрузки уровней (Levels.hs)

Модуль ``Levels.hs`` отвечает за хранение и загрузку базовых головоломок программы. В нём уровни разделены по сложностям Easy, Medium и Hard. Каждая головоломка хранится в компактном строковом виде длины 81 символ, соответствующем клеткам поля 9x9. Заполненные клетки задаются цифрами от 1 до 9, а пустые - символом 0.

Основной функцией модуля является преобразование строкового представления в структуру ``LoadedPuzzle``. Эта структура содержит игровую доску и информацию о том, какие клетки являются исходными и не могут изменяться пользователем.

Такое решение было выбрано потому, что оно упрощает хранение уровней, делает код компактнее и позволяет легко добавлять новые головоломки.

Пример записи доски с помощью одной строки:

```
"53007000060019500009800006080006000340080300170002000606000028000  
0419005000080079"
```

Реализация представления доски (Board.hs)

Здесь реализуются правила игры. В данном модуле можно выделить основные функции:

- `boardGet` - получение значения клетки;
- `boardSet` - установка значения в клетку;
- `boardClear` - очистка клетки;
- `cellToRC` и `rcToCell` - преобразование индекса клетки в строку и столбец и обратно;
- `rowIndices`, `colIndices`, `boxIndices` - получение клеток строки, столбца и блока;
- `candidatesAt` - вычисление допустимых кандидатов для клетки;
- `conflictingCells` - поиск конфликтующих клеток;
- `checkMove` - проверка корректности хода;
- `applyMove` - применение пользовательского хода;
- `isSolved` - проверка, решено ли sudoku.

Реализация генерации новых головоломок (Generator.hs)

Данный модуль отвечает за то, чтобы пользователь при желании мог сгенерировать новую головоломку вместо загрузки её из файла. При этом, стоит отметить, что выполняется не полная генерация новой головоломки с нуля, а псевдо-генерация.

Если начать процесс генерации с нуля, он затянется на слишком длительное время. Поэтому было принято решение опираться на случайную головоломку для каждой сложности, заранее предустановленную, и выполнять случайные преобразования внутри неё, сохраняя их уникальность. Внутри корректной головоломки выполняются перестановки таким образом, чтобы она оставалась корректной (не появлялись конфликты, сохранялось единственное решение).

Можно выделить основные функции:

- `generatePuzzle` - генерация новой головоломки заданной сложности;
- `difficultyPuzzles` - выбор базовых головоломок подходящей сложности;
- `matchesDifficulty` - проверка соответствия уровня заданной сложности;
- `transformPuzzle` - безопасное преобразование базовой головоломки;
- `permuteDigits` - перестановка цифр;
- `permuteRowsWithinBands` - перестановка строк внутри блоков;
- `permuteColsWithinStacks` - перестановка столбцов внутри блоков.

Дополнительно базовые уровни были подобраны так, чтобы соответствовать своим уровням сложности с точки зрения встроенного решателя (будет описан ниже):

- ``Easy`` решается ``NakedSingle``;
- ``Medium`` требует как минимум ``HiddenSingle``;
- ``Hard`` требует более сильного алгоритма (например, `Backtracking`).

Благодаря этому кнопка ``Generate`` для каждой сложности действительно создаёт задачи соответствующего класса.

Как выполняется рандомизация

В первую очередь, случайным образом создается новая перестановка для цифр, например:

$$[1, 2, 3, 4, 5, 6, 7, 8, 9] \rightarrow [4, 1, 9, 2, 7, 3, 6, 5, 8]$$
$$(1 \rightarrow 4; 2 \rightarrow 1; \dots 9 \rightarrow 8)$$

Если заменить все цифры в головоломке по такому правилу, её корректность сохранится.

Далее строим разбиение строк доски по блокам (1..3, 4..6, 7..9), в каждом блоке случайным образом перемешиваются строки. Затем переставляются сами блоки случайным образом.

После перестановки строк аналогичная операция выполняется для всех столбцов с аналогичным разбиением доски.

Таким образом, после данных преобразований сгенерируется новая головоломка, для которой сохранится корректность, ведь на каждом из этапов не образуются новые конфликты.

Реализация отрисовки интерфейса (UI.hs)

Модуль UI отвечает за отрисовку интерфейса программы.

Основные функции:

- `renderWorld` - главная функция отрисовки;
- `drawMainMenu` - отрисовка главного меню;
- `drawDifficultyMenu` - отрисовка экрана выбора сложности;
- `drawPuzzleMenu` - отрисовка экрана выбора головоломки;
- `drawGameWorld` - отрисовка игрового экрана;
- `drawBoard` - отрисовка игрового поля;
- `drawDigits` - отрисовка цифр на доске;
- `drawSelection` - отрисовка выделенной клетки;
- `drawRelatedUnits` - подсветка строки, столбца и блока выбранной клетки;
- `drawHoverCell` - подсветка клетки под курсором;
- `drawConflicts` - подсветка конфликтов;
- `drawTopBar` - верхняя панель кнопок;
- `drawStatusBar` - строка состояния;
- `drawButtonOnScreen` - отрисовка кнопки;
- `buttonRectForScreen` - координаты и размеры кнопок;
- `buttonAt` - определение кнопки по координатам мыши;
- `pointToCell` - определение клетки по координатам мыши.

Стоит рассмотреть ключевой принцип построения интерфейса.

Рассмотрим функцию

```
drawMainMenu world =
```

Pictures

```
[ drawBackground world
, drawMenuTitle world 0.26 "Sudoku"
, drawMenuSubtitle world "Choose a puzzle & test your logic!"
, drawButtonOnScreen world MainMenu ButtonPlay
, drawButtonOnScreen world MainMenu ButtonTheme
]
```

В данном случае используется функция `Pictures` из библиотеки `Gloss`. Она позволяет «склеить» все элементы массива в единый экран. Таким образом, были построены все экраны для отрисовки в программе. При различных состояниях `World` отрисовываются соответствующие экраны, представленные как массив из «картинок», с помощью функции `renderWorld`.

Реализация обработки событий в приложении (Events.hs)

Модуль `Events` реализует поведение программы и связывает интерфейс с логикой.

Основные функции:

- `handleEvent` - главный обработчик событий;
- `handleClick` - обработка кликов мыши;
- `handleButton` - реакция на нажатие кнопок;
- `handleCharacter` - обработка ввода цифр;
- `handleSpecialKey` - обработка специальных клавиш;
- `updateWorld` - обновление состояния программы по времени;
- `updateUIEffects` - обновление визуальных эффектов;
- `showHint` - показ следующего хода;
- `startSolverAnimation` - запуск автоматического решения;
- `advanceSolveScript` - выполнение следующего шага анимации;
- `runNextSolveAction` - применение очередного действия решателя;
- `pickRandomPuzzle` - выбор случайной головоломки;
- `resetTransientUI` - сброс временных интерфейсных эффектов.

Обработчик событий связывает пользовательские действия с изменением глобального состояния ``World``.

Ниже указаны примеры использования функций:

- при клике на кнопку ``Play`` происходит переход к экрану выбора сложности;
- при выборе сложности происходит переход к экрану выбора конкретной головоломки;
- при выборе уровня создаётся новое игровое состояние;
- при клике по клетке она выделяется;
- повторный клик по той же клетке снимает выделение.

Таким образом, пользователь может удобно взаимодействовать с миром, быстро переключаясь между нужными состояниями.

Реализация решателя головоломок (Solver.hs)

Существует множество стратегий решения sudoku: как те, которые используются самим человеком, так и те, которые способны решить практически любую головоломку, но с помощью перебора всех возможных вариантов. В проекте были реализованы четыре стратегии решения:

1. ``NakedSingle``
2. ``HiddenSingle``
3. ``NakedPair``
4. ``Backtracking``

Ниже были приведены краткие описания каждой из стратегий:

- Стратегия ``NakedSingle`` ищет пустую клетку, у которой есть ровно один возможный кандидат.
- Стратегия ``HiddenSingle`` рассматривает строку, столбец или блок и определяет, может ли некоторая цифра стоять только в одной клетке данной области.
- Стратегия ``NakedPair`` ищет две клетки с одинаковой парой кандидатов в одной области и использует это ограничение для вывода следующего хода.
- Стратегия ``Backtracking`` запускает рекурсивный перебор с возвратом. Она используется тогда, когда более простые стратегии уже не дают результата.

В реализации модуля можно выделить следующие ключевые функции:

- `solveStep` - поиск одного следующего хода;
- `solveWithStrategy` - решение доски с ограничением по выбранной стратегии;
- `solveFull` - полное решение sudoku;
- `strategyLabel` - текстовое имя стратегии;
- `solverStrategies` - список доступных стратегий.

При этом, стоит отметить, что стратегия `Backtracking` была оптимизирована: на каждом шаге рекурсии выбиралась клетка поля, у которой минимальное число возможных кандидатов (функция `bestBranchCell`). Таким образом, на каждом этапе рекурсии вызывалось минимальное число новых функций. Это позволило сильно сократить время перебора для нахождения верного решения.

В программе решатель применяется двумя способами: либо решатель находит следующий корректный ход, который используется в функции подсказки, либо пользователь может запустить решатель для полного автоматического решения sudoku при нажатии на кнопку ``Solve``. При этом в процессе автоматического решения в поле состояний показывается последовательный перебор стратегий, который завершится по мере нахождения максимально простой и подходящей.

Реализация ассетов и тем оформления (Assets.hs)

Для улучшения интерфейса в программу были добавлены иконки:

- лампочка для подсказки;
- значок выбора темы;
- кубик для кнопки ``Random level``;
- стрелки обновления.

В зависимости от темы используются обычные (чёрные) или белые версии ассетов. Это позволяет сохранять хорошую читаемость и на светлом, и на тёмном фоне.

Часть цветов интерфейса была вынесена в ``Config.hs``, чтобы упростить изменение темы, уменьшить количество лишних констант в ``UI.hs``, а также чтобы сделать код более чистым.

При этом локальные небольшие текстовые смещения оставлены в ``UI.hs``, поскольку они относятся не к глобальной конфигурации приложения, а к конкретной настройке отрисовки отдельных кнопок.

Тестирование программы

Проверка программы выполнялась по нескольким направлениям. Ниже перечислены ключевые аспекты тестирования программы, по разделам.

Проверка сборки

Проект собирается с помощью ``cabal build``. В процессе сборки не возникает ни одного предупреждения.

Проверка интерфейса

В первую очередь проверялись переходы между экранами. При каждой смене состояния экрана отступы были настроены таким образом, чтобы не возникало «скачков» между различными положениями текста и кнопок. Удалось добиться их минимизации.

Также проверялись такие составляющие, как:

- корректная работа всех кнопок;
- смена темы;
- корректность подсветок (при взаимодействии с интерфейсом);
- корректное отображение иконок.

Особое внимание уделялось экрану выбора уровней и игровому экрану, поскольку именно там сосредоточено большинство визуальных элементов.

Проверка игровой логики

Проверялись следующие моменты:

- невозможность менять исходные клетки;
- корректность ввода цифр;
- корректность удаления значения;
- обнаружение конфликтов;
- завершение игры при правильном заполнении.

Все случаи обрабатывались корректно и были учтены.

Проверка подсказки и решателя

В данном разделе проверялись:

- корректность нахождения следующего хода;
- запуск автоматического решения;
- поочерёдная анимация заполнения;
- блокировка пользовательского ввода во время автоматического решения.

Следующий ход был действительно верным, анимация корректно отрабатывала при запуске кнопки, решатель заполнял поле до конца без конфликтов на доске, при этом пользователь не мог вводить собственные ходы во время автоматического решения.

Проверка генерации

Отдельно проверялась генерация задач по сложности. Для этого анализировалось, решаются ли сгенерированные уровни теми стратегиями, для которых они предназначены.

В результате была доработана база исходных уровней, чтобы генерация различалась по классам ``Easy``, ``Medium`` и ``Hard``.

Проверка некорректных ситуаций

Кроме проверки корректной работы программы, отдельно анализировались ситуации, связанные с ошибочными и граничными состояниями.

В программе предусмотрены и были проверены следующие случаи:

- попытка поставить цифру в исходную неизменяемую клетку;
- попытка сделать ход, нарушающий правила sudoku;
- попытка обратиться к несуществующей клетке;
- отсутствие доступной подсказки для текущего состояния доски;
- невозможность решить текущую доску встроенным решателем;
- отсутствие доступных головоломок при случайном выборе;
- отсутствие конкретного уровня или выбранной головоломки;
- аварийный случай, при котором генератор не находит подходящих базовых пазлов.

Для таких ситуаций в программе предусмотрены защитные механизмы. Например, при некорректном ходе пользователю выводятся сообщения о том, что ход недопустим или клетка не может быть изменена. При отсутствии подсказки отображается сообщение ``No hint available``. Если ни одна стратегия не может решить текущую доску, выводится сообщение ``No solver could solve this board``.

Отдельно предусмотрен безопасный резервный сценарий для генератора. Если по какой-либо причине не удаётся получить список подходящих базовых головоломок, программа не завершается с ошибкой, а использует запасную корректную доску, встроенную в модуль генерации. Аналогично, при проблеме загрузки встроенных уровней в начальном состоянии формируется сообщение ``Load error``, что позволяет явно отобразить ошибку вместо аварийного завершения приложения.

Проверка таких состояний показала, что программа корректно обрабатывает ошибочные ситуации, не завершается аварийно и сохраняет работоспособность интерфейса.

Визуализация игрового процесса



Главный экран приложения. Есть кнопка Play и смена темы. Смена темы доступна и в следующих экранах. Иконка была реализована с помощью загрузки ассетов из папки `/assets`.

Choose a difficulty

Easy

Medium

Hard



Back

После нажатия кнопки Play предлагается выбор сложности. Справа снизу добавляется кнопка Back для перехода на 1 экран назад.

При наведении курсора на кнопку её фон становится темнее.

Medium puzzles

LOAD:

1	2	3	4
5	Random level 		

OR:

Generate 
--



Back

После выбора сложности предлагается выбрать 1-5 уровень, либо случайный от 1 до 5. Также снизу есть возможность сгенерировать новый случайный уровень.

Значки у кнопок были получены с помощью загрузки ассетов из папки `/assets`


Solve


Menu

			3		1		5	
	5	7	4	6	9			8
9	6			2	5	3	7	
5			9	3			8	
		3						
	9		5		6			2
		5	6	9	8			
6	7	9		4				5
	4	2	1		7		6	3

Sudoku
Moves: 0

После нажатия любой кнопки с выбором уровня (в данном случае, Generate) открывается окно с выбранным уровнем.

При наведении курсора на любую клетку она подсвечивается легким голубым цветом.

На экране можно видеть состояние слева снизу (в данном случае, состояние по умолчанию, надпись “Sudoku”), а также количество ходов.

Сверху доступны опции «подсказка», запуск автоматического решателя, выбор темы, сброс уровня, возврат в меню.



Solve



Menu

			3		1		5	
	5	7	4	6	9			8
9	6			2	5	3	7	
5			9	3			8	
		3						
	9		5		6			2
		5	6	9	8			
6	7	9		4				5
	4	2	1		7		6	3

Sudoku

Moves: 0

При нажатии на клетку она подсвечивается более ярким цветом, а также подсвечиваются строка, столбец и блок, которые ей соответствуют.



Solve



Menu

			3		1		5	
	5	7	4	6	9			8
9	6			2	5	3	7	
5			9	3			8	
		3						
	9		5		6			2
		5	6	9	8			
6	7	9		4				5
	4	2	1		7		6	3

This move is not allowed

Moves: 0

Если ввести ход, который вызывает конфликты (например, цифру 2) – клетка подсвечивается красным цветом, который затухает. При этом слева снизу обновляется состояние.



Solve



Menu

			3		1		5	
	5	7	4	6	9			8
9	6			2	5	3	7	
5			9	3			8	
		3						
	9		5		6			2
		5	6	9	8			
6	7	9		4				5
	4	2	1		7		6	3

Hint: place 8

Moves: 0

После запуска подсказки предлагается следующий ход.



Solve



Menu

2	8	4	3	7	1	6	5	9
3	5	7	4	6	9	1	2	8
9	6	1	8	2	5	3	7	4
5	2	6	9	3	4	7	8	1
4	1	3	7	8	2	5	9	6
7	9	8	5	1	6	4	3	2
1	3	5	6	9	8	2	4	7
6	7	9	2	4	3	8	1	5
8	4	2	1	5	7	9	6	3

Solved using HiddenSingle strategy

Moves: 42

После запуска «Solve» поле заполняется, подсвечивается зелёным цветом, слева снизу пишется, с помощью какой стратегии удалось решить sudoku.



Solve





Menu

			3		1		5	
	5	7	4	6	9			8
9	6			2	5	3	7	
5			9	3			8	
		3						
	9		5		6			2
		5	6	9	8			
6	7	9		4				5
	4	2	1		7		6	3

Sudoku

Moves: 0

Доступна смена темы, а также сброс головоломки.

Sudoku

Choose a puzzle & test your logic!

Play



Кнопка Menu возвращает пользователя на главный экран.

Использование технологий искусственного интеллекта

В ходе работы использовались версии ChatGPT 5.4 и 5.5, для обсуждения ключевых моментов работы программы.

Обсуждение структуры программы

Изначально самостоятельно был создан проект, в котором существовали модули ``Main.hs``, ``Types.hs``, ``Board.hs``, ``Levels.hs``, ``UI.hs``. В процессе обсуждения задания с LLM было решено дополнительно реализовать оставшиеся модули, которые бы улучшили программу.

Был задан следующий промпт:

«Изучи текущую структуру проекта, прочитай условие задания. Предложи на основании моей текущей структуры итоговую финальную структуру проекта и план реализации модулей».

После чего, опираясь на ответ LLM, были созданы дополнительные модули, которые сейчас существуют в проекте. Удалось обсудить, каким образом лучше разделить работу программы, какие названия файлов создать и что в них должно быть.

Обсуждение основных стратегий решения

Был задан промпт:

«Предложи ключевые стратегии решения головоломок, которых будет достаточно для проекта, объясни их суть»

Таким образом, была получена идея реализовать 4 стратегии решения, которые сейчас существуют в проекте.

В процессе реализации модель использовалась для обсуждения, как улучшить самостоятельно реализованные стратегии решения. Например, LLM предложила реализовать функцию для нахождения клетки с минимальным числом кандидатов для оптимизации работы `Backtrack`.

Промпт:

«Заметил, что при использовании `backtracking` приложение слишком сильно тормозит, вычисления происходят слишком долго. Как можно улучшить функцию? Предложи улучшения».

После проверки ответов LLM, изменения вносились в основной код.

Обсуждение работы интерфейса

В первую очередь, задавался вопрос:

«Каким образом в моём приложении реализовать смену разных окон? Предложи идею, как это можно максимально просто реализовать, используя библиотеку `Gloss`».

Далее была реализована идея: изменять состояния мира, а именно, состояние “uiWorld”, и отрисовывать тот интерфейс, название которого указано в состоянии. Отрисовка происходила с помощью функции Picture, которая была рассмотрена выше.

Рефакторинг кода

После финальной реализации, были заданы промпты:

«Изучи код <UI.hs>. Предложи вариант, что можно вынести в конфиг, а что оставить как есть?»

После анализа ответа ChatGPT удалось добиться баланса между вынесенными в конфиг параметрами и локальными параметрами внутри каждой программы.

Помощь в улучшении генерации уровней

Задавался вопрос:

«Сейчас уровни сложности Easy порой решаются не с помощью NakedSingle, а с помощью HiddenSingle. А некоторые уровни из Hard решаются NakedSingle. Почему происходит такое несовпадение и как это исправить?»

После анализа ответа ChatGPT и собственной перепроверки удалось понять, что проблема не в реализованных функциях, а в самих загруженных головоломках. Он их исправил таким образом, что теперь они действительно решаются соответствующими стратегиями.

Головоломки корректные, поскольку решатель находит единственно верное решение.

Подытоживая работу с искусственным интеллектом, можно сделать вывод, что это достаточно мощный инструмент для работы, который позволил сократить время при перепроверке реализованных функций, а также он позволил быть более уверенным в придуманной финальной архитектуре проекта.

При продуманной архитектуре сложность работы сильно сокращается, поэтому до начала работы было важно продумать её до мелочей.

Заключение

В ходе выполнения работы была создана программа «Sudoku» на языке Haskell с графическим интерфейсом и набором дополнительных возможностей.

В программе были реализованы:

- многоэкранный интерфейс;
- выбор сложности;
- выбор конкретного уровня;
- случайная загрузка готовой головоломки;
- генерация новых вариантов;
- подсказка следующего хода;
- автоматическое решение;
- светлая и тёмная тема;
- визуальные эффекты и подсветки.

Также в проекте была продумана архитектура:

- данные, логика, интерфейс и конфигурация разделены по модулям;
- код структурирован и пригоден для дальнейшего расширения;
- приложение находится в рабочем и завершённом состоянии.

Поставленная цель была достигнута. Все основные задачи, сформулированные в начале работы, были выполнены.

Литература

1. Haskell Language. Official Website. URL: <https://www.haskell.org/>
2. Hoogle: Haskell API Search. URL: <https://hoogle.haskell.org/>
3. Gloss Library Documentation ('Graphics.Gloss'). Hackage. URL: <https://hackage.haskell.org/package/gloss/docs/Graphics-Gloss.html>
4. Cabal User Guide. URL: <https://cabal.readthedocs.io/>
5. Data.Vector package documentation. Hackage. URL: <https://hackage.haskell.org/package/vector>
6. containers package documentation ('Data.Set'). Hackage. URL: <https://hackage.haskell.org/package/containers>
7. The Glasgow Haskell Compiler User's Guide. URL: https://downloads.haskell.org/ghc/latest/docs/users_guide/
8. Sudoku rules and solving techniques. Conceptis Puzzles. URL: <https://www.conceptispuzzles.com/index.aspx?uri=puzzle/sudoku/techniques>